

PFE-M 2026 : Big Data

LIU - Master

Consignes Générales : Groups de **2 max.** Un seul sujet au choix sur le dataset **NYC Yellow Taxi**. L'évaluation porte sur le code ET l'analyse de performance (CPU/RAM/IO).

1 Introduction

Ce projet valide votre capacité à manipuler des volumes massifs sous contraintes. En utilisant HDFS et Spark, vous devez répondre aux enjeux de la **Mauritanie** (SMELEC, SNIM, SNDE, etc.).

2 Sujets au choix

Sujet 1 L'Architecte du Stockage

Compétence : Optimisation (Parquet vs CSV). Challenge : Réduire la taille SNIM de 80% et booster la vitesse x10. → *Détails Annexe A*

Sujet 2 Le Maître de l'Algorithme

Compétence : Calcul Distribué (MapReduce). Challenge : Décoder la rentabilité du transport national heure par heure. → *Détails Annexe B*

Sujet 3 Le Gardien de la Donnée

Compétence : Qualité (Data Cleaning). Challenge : Créer le filtre anti-erreurs de facturation pour la SMELEC. → *Détails Annexe C*

Sujet 4 L'Analyste Stratégique

Compétence : Intelligence Croisée (Shuffle Joins). Challenge : Corréler climat et économie via des jointures massives. → *Détails Annexe D*

Sujet 5 L'Ingénieur Système

Compétence : Design Physique (Partitioning). Challenge : Structurer le Recensement National pour des accès instantanés. → *Détails Annexe E*

"Annexes..."

A Détails : Sujet 1 (Stockage)

Ingénierie du Stockage (Colonnaire vs Ligne)

Défi : Optimisation de la sérialisation et efficience de la bande passante HDFS.

- **Contexte Technique** : Le stockage en ligne (CSV) impose une lecture séquentielle (*Row-based*) qui sature le réseau lors des analyses analytiques. Ce projet explore le passage au stockage colonnaire (*Column-oriented*) pour minimiser l'I/O disque et optimiser le *Column Pruning*.
- **T.A.F** :
 - Ingestion du dataset *NYC Taxi* (10 Go) depuis HDFS.
 - Transformation et réécriture des données sous format **Apache Parquet** en testant trois algorithmes de compression : **Snappy** (équilibre), **Gzip** (haute compression) et **Uncompressed**.
 - Analyse du **Data Skew** (déséquilibre de charge) : Vérifier si les fichiers sont répartis équitablement entre les *DataNodes* ou si certains blocs créent des goulots d'étranglement.
- **Indicateurs de Performance (KPIs)** :
 - Comparaison du **Ratio de Compression** (Taille CSV vs Taille Parquet).
 - Benchmark de vitesse : Temps d'exécution d'une requête complexe (ex : *SELECT AVG(fare_amount) WHERE vendor_id = 1*) via `spark.time()`.
 - Analyse du plan d'exécution (`df.explain()`) pour prouver le saut de colonnes inutiles.

Dans un pays où les infrastructures réseau entre les villes (ex : Nouakchott - Zouerate) peuvent être limitées, l'économie de bande passante est une priorité nationale.

- **Cas SNIM** : Le passage au format Parquet pour les logs de maintenance permettrait de réduire les coûts de stockage de 70% tout en accélérant l'accès aux données historiques.
- **Cas SMELEC** : Pour l'analyse de millions de relevés, le stockage colonnaire permet d'interroger uniquement la colonne "Consommation" sans lire les colonnes "Identifiant" ou "Localisation", divisant ainsi le temps de traitement par dix.

L'étudiant doit démontrer que le choix d'un format de fichier n'est pas un détail technique, mais une décision économique majeure pour la souveraineté numérique du pays.

B Détails : Sujet 2 (MapReduce)

Paradigmes de Calcul (MapReduce Engine)

Défi : Analyse de rentabilité par micro-segmentation horaire via le moteur de calcul natif Hadoop.

- **Contexte Technique** : L'abstraction des DataFrames Spark masque souvent la complexité du mouvement des données. Ce sujet impose un retour aux sources avec le paradigme **MapReduce**. L'objectif est de manipuler des paires *Clé-Valeur* pour effectuer un calcul de rentabilité (Ratio : Recette / Kilométrage) sans l'aide de l'optimiseur Catalyst de Spark.
- **T.A.F** :
 - **Phase Map** : Extraire l'heure de prise en charge (*pickup_hour*) comme clé, et émettre un tuple complexe (*fare_amount*, *trip_distance*) comme valeur.

- **Phase Shuffle et Sort** : Documenter le mécanisme de partitionnement par défaut (HashPartitioner) et expliquer comment les données de même clé sont regroupées physiquement.
- **Phase Reduce** : Implémenter l'agrégation pour calculer la moyenne pondérée du gain par kilomètre pour chaque tranche horaire.
- **Exigences de Réflexion** :
 - **Analyse du Shuffle** : Expliquer mathématiquement le volume de données transitant sur le réseau. Si N est le nombre de lignes et R le nombre de Reducers, quel est l'impact de R sur la parallélisation et sur le temps de fusion (*Merge*) ?
 - **Gestion du Data Skew** : Que se passe-t-il si 80% des trajets ont lieu à 8h du matin ? Analyser l'impact sur le Reducer responsable de cette clé.

La maîtrise du MapReduce permet de concevoir des systèmes de facturation et de logistique robustes, capables de fonctionner sur des clusters hétérogènes.

- **Optimisation des transports** : Pour une entreprise de transport reliant Nouakchott à Nouadhibou, cette analyse permettrait d'identifier les fenêtres horaires les plus rentables en croisant la consommation de carburant et les tarifs appliqués.
- **SMELEC (Analyse de charge)** : Utiliser la logique MapReduce pour agréger les consommations par transformateur de quartier. Le "Reducer" devient ici l'unité de calcul qui consolide la charge d'un secteur spécifique de la ville.

L'étudiant doit conclure sur la pertinence du MapReduce pour les calculs "Batch" massifs où la tolérance aux pannes et la prévisibilité du mouvement des données priment sur la vitesse pure.

C Détails : Sujet 3 (Cleaning)

Intégrité et Data Cleaning Distribué

Défi : Conception d'un moteur de validation de données (*Data Quality Framework*) à l'échelle du Téraoctet.

- **Contexte Technique** : Dans un environnement Big Data, la qualité de la donnée est le premier verrou de la décision. Ce sujet traite de l'implémentation de filtres statistiques distribués pour nettoyer un flux massif. L'étudiant doit dépasser le simple filtrage manuel pour mettre en place une détection d'anomalies basée sur la distribution statistique des données (Loi Normale).
- **T.A.F** :
 - **Profilage des Données** : Calculer les métriques de base (Moyenne, Écart-type, Quartiles) sur l'ensemble du dataset *NYC Taxi* en une seule passe Spark.
 - **Détection d'Outliers** : Implémenter la méthode du **Z-Score** ($Z = \frac{x-\mu}{\sigma}$) pour identifier les valeurs aberrantes sur les colonnes critiques (*fare_amount*, *trip_distance*).
 - **Traitement des "Dirty Data"** : Gérer les incohérences logiques (ex : vitesse moyenne du taxi supérieure à 200 km/h) et les valeurs manquantes via des stratégies d'imputation ou de suppression.
- **Exigences de Réflexion** :
 - **Impact sur les Agrégats** : Comparer la variance et la moyenne des recettes avant et après nettoyage. L'étudiant doit quantifier l'erreur statistique induite par la non-qualité.

- **Coût du Rejet** : Calculer le *Taux de Rejet* global et analyser si les données rejetées sont concentrées sur certains capteurs ou périodes (biais de sélection).

Le nettoyage des données est une étape vitale pour la confiance numérique entre l'État et le citoyen.

- **Fiabilisation SMELEC / SNDE** : L'application de ce pipeline permettrait de bloquer automatiquement la facturation en cas de lecture aberrante (ex : un compteur domestique affichant 5000 kWh en une journée suite à un bug technique). Cela réduit les réclamations et améliore l'image de marque de la société.
- **Statistiques Nationales (ANSADE)** : Garantir que les indicateurs économiques calculés à partir des données de terrain ne sont pas faussés par des erreurs de saisie ou des pannes de transmission réseau.

L'étudiant doit démontrer qu'un bon Data Engineer ne se contente pas de traiter la donnée, il en garantit la véracité pour éviter le phénomène du "*Garbage In, Garbage Out*".

C.1 Sujet 4 : Enrichissement Multi-Source (Shuffle Joins)

Défi : Corrélation entre variables exogènes et indicateurs d'activité via des jointures distribuées optimisées.

- **Contexte Technique** : En Big Data, la valeur réside dans le croisement des sources. Cependant, la jointure (*Join*) est l'opération la plus coûteuse car elle déclenche souvent un *Network Shuffle* (déplacement massif de données entre les nœuds). Ce sujet explore les stratégies de jointure de Spark pour minimiser la latence réseau lors de l'enrichissement d'un dataset transactionnel massif par un dataset référentiel.
- **T.A.F** :
 - **Fusion de Données** : Croiser le dataset *NYC Taxi* (Fact Table) avec un dataset météo (Dimension Table) sur la clé temporelle (Date/Heure).
 - **Analyse Comparative des Algorithmes** :
 1. Forcer un **SortMergeJoin** et observer l'étape de *Shuffle Exchange*.
 2. Implémenter un **BroadcastHashJoin** (en utilisant `broadcast()`) pour les petites tables et mesurer le gain de performance.
 - **Analyse de Corrélation** : Extraire des insights : "L'intensité des précipitations augmente-t-elle significativement le temps de trajet moyen ou le montant des pourboires?"
- **Exigences de Réflexion** :
 - **Exploration du Plan d'Exécution** : Utiliser `df.explain()` pour identifier les opérateurs physiques. Expliquer pourquoi Spark choisit l'un ou l'autre par défaut selon le paramètre `spark.sql.autoBroadcastJoinThreshold`.
 - **Problématique du "Small Table"** : Que se passe-t-il si la table météo devient trop grande pour tenir en mémoire sur un seul exécuteur?

Le croisement de données hétérogènes est le moteur de l'aide à la décision stratégique pour les grands projets nationaux.

- **Secteur Agricole (Vallée du Fleuve)** : Croiser les données de production agricole avec les données météorologiques et de niveau de crue pour prédire les rendements et optimiser les périodes de récolte.

- **Énergie (SMELEC)** : Corréler la consommation électrique avec les vagues de chaleur enregistrées par les stations météo nationales. Cette jointure permet de planifier les pics de charge et d'éviter les délestages préventifs.
- **Logistique minière (SNIM)** : Croiser les incidents techniques sur la voie ferrée avec les conditions de vents de sable pour adapter la vitesse des convois.

L'étudiant doit démontrer qu'une jointure bien optimisée permet d'obtenir une "Intelligence Métier" profonde sans saturer les infrastructures de communication du pays.

D Détails : Sujet 5 (Partitioning)

Architecture de Partitionnement (Physical Design)

Défi : Optimisation du *Partition Pruning* et structuration physique des données pour les requêtes à haute sélectivité.

- **Contexte Technique** : Sur un système de fichiers distribué comme HDFS, la performance dépend de la capacité du moteur de calcul à éviter la lecture de données inutiles. Le partitionnement crée une hiérarchie de dossiers physiques permettant à Spark de pratiquer l'élagage de partitions (*Partition Pruning*), une technique fondamentale pour réduire radicalement le *Scanning* du disque.
- **T.A.F** :
 - **Restructuration HDFS** : Charger le dataset *NYC Taxi* et le sauvegarder en utilisant la clause `partitionBy()` de Spark.
 - **Stratégie Multi-Niveaux** : Implémenter un partitionnement hiérarchique (ex : par *VendorID* puis par *Month*). Observer la structure générée dans HDFS : `/data/taxi/VendorID=1/`
 - **Benchmark I/O** : Exécuter une requête filtrée sur les colonnes de partition et comparer les métriques "*File Scan Bytes Read*" dans l'interface Spark UI entre le dataset brut (CSV/Parquet plat) et le dataset partitionné.
- **Exigences de Réflexion** :
 - **Problématique de la Cardinalité** : Discuter du risque d'*Over-partitioning* (trop de petites partitions). Pourquoi avoir 10 000 fichiers de 10 Ko est-il catastrophique pour le NameNode de Hadoop ?
 - **Taille de Bloc vs Taille de Partition** : Analyser le rapport entre la taille de vos partitions sur disque et la taille des blocs HDFS (par défaut 128 Mo).

Le partitionnement est le pilier de la souveraineté et de l'efficacité des infrastructures de données nationales.

- **Secteur Public (État Civil)** : Partitionner les données du recensement national par **Wilaya** et par **Moughataa**. Cela permet au ministère de l'Intérieur d'extraire les statistiques d'une commune spécifique en quelques secondes sans scanner les données de tout le pays.
- **SMELEC (Monitoring Régional)** : Structurer les données de consommation par région. Si une analyse est demandée pour la ville de Nouadhibou, le système ignore physiquement les fichiers de Nouakchott ou de Rosso, préservant ainsi les ressources du cluster central.
- **SNIM (Gestion du Matériel)** : Organiser les données de capteurs par type de machine (Locomotives, Dragues, Camions). L'analyse d'une panne sur une machine précise devient instantanée malgré des années d'historique.

L'étudiant doit démontrer qu'un partitionnement intelligent transforme un "Vrac de données" en un système d'information performant, capable d'évoluer avec la croissance démographique et industrielle de la Mauritanie.